

Spring Boot JPA ja Spring Transaction -raportti

Mitä sovelluksessa tapahtuu?

Sovellus toimii yksinkertaisena pankkijärjestelmänä, jossa käyttäjä voi siirtää rahaa tilien välillä. Etusivulla näytetään lista kaikista tileistä ja niiden saldoista.

Kun käyttäjä painaa *Send Money* -painiketta, hän siirtyy lomakkeelle, jossa valitaan lähettäjätili, vastaanottajtili ja siirrettävä summa. Lomakkeen lähetyksen jälkeen sovellus tarkistaa, onko lähettäjätilillä riittävästi katetta.

Jos rahaa on tarpeeksi, summa vähennetään lähettäjän tililtä ja lisätään vastaanottajan tilille. Jos taas katetta ei ole tai jokin muu virhe tapahtuu, koko tapahtuma peruutetaan. Tällöin mitään muutoksia ei tallenneta tietokantaan, vaan tilien saldot pysyvät ennallaan.

Tiedostojen roolit

SpringBootApplication.java

Sovelluksen käynnistyspiste. @SpringBootApplication huolehtii sovelluksen automaattisesta konfiguroinnista.

application.properties

Konfiguraatiotiedosto, jossa määritellään esimerkiksi tietokantayhteys (MySQL), Hibernate-asetukset ja muut tarvittavat asetukset.

BankAccount.java

JPA-entiteetti, joka vastaa tietokannan *bank_account*-taulua. Annotaatiot kuten @Entity, @Table ja @Column määrittelevät, miten luokka vastaa tietokannan rakennetta.

BankAccountInfo.java

Tavallinen Java-luokka (DTO), jota käytetään hakemaan tietoa kannasta JPQL-kyselyllä. Ei ole varsinainen entiteetti.

BankTransactionException.java

Oma poikkeusluokka, jota käytetään tilanteissa, joissa siirtoa ei voida suorittaa (esimerkiksi tilillä ei ole katetta). Tämän avulla voidaan myös laukaista transaktion rollback.

BankAccountDAO.java

Tietokantakerros (DAO), joka käyttää EntityManageria tietokantaoperaatioihin. @Repository-annotaatio rekisteröi sen Spring-komponentiksi.

addAmount()-metodi on merkitty @Transactional(propagation = Propagation.MANDATORY), mikä tarkoittaa, että sitä saa kutsua vain olemassa olevan transaktion sisällä.

BankAccountService.java

Palvelukerros, jossa varsinainen liiketoimintalogiikka sijaitsee.

sendMoney()-metodi on merkitty @Transactional-annotaatiolla, jolloin koko rahansiirto suoritetaan yhtenä transaktiona.

MainController.java

Controller, joka käsittelee HTTP-pyynnöt ja välittää ne palvelukerrokseen. Lisäksi se toimittaa datan näkymille.

index.html ja sendMoney.html

Thymeleaf-templatteja, jotka muodostavat käyttöliittymän. Ensimmäinen näyttää tilit ja toinen siirtolomakkeen.

Miten @Transactional toimii?

@Transactional on Springin tarjoama annotaatio, jonka avulla hallitaan tietokantatransaktioita. Sen toiminta perustuu AOP:hen (Aspect-Oriented Programming) ja proxy-olioihin.

Käytännössä Spring luo annotoidun luokan ympärille ns. proxy-olion. Kun metodia kutsutaan, kutsu menee ensin tämän proxyn kautta, joka hoitaa transaktion hallinnan.

Transaktion kulku menee näin:

- Transaktio aloitetaan (BEGIN)
- Varsinainen metodi suoritetaan
- Jos kaikki onnistuu → COMMIT (muutokset tallennetaan)
- Jos tapahtuu virhe → ROLLBACK (kaikki muutokset perutaan)

Transaktiot tässä sovelluksessa

BankAccountService.sendMoney()

Tämä metodi käynnistää transaktion. Se suorittaa rahansiirron kahdessa vaiheessa: vähentää rahaa lähettäjältä ja lisää vastaanottajalle.

Jos jossain vaiheessa tapahtuu virhe (esim. BankTransactionException), koko transaktio perutaan.

BankAccountDAO.addAmount()

Tämä metodi ei aloita omaa transaktiota, vaan liittyy jo olemassa olevaan. Tämä varmistetaan propagation = Propagation.MANDATORY -asetuksella.

Jos metodia kutsutaan ilman aktiivista transaktiota, Spring heittää poikkeuksen.

Tärkeimmät @Transactional-parametrit

propagation

Määrittelee, miten metodi toimii suhteessa olemassa olevaan transaktioon:

- REQUIRED (oletus): liittyy olemassa olevaan tai luo uuden
- MANDATORY: vaatii olemassa olevan
- REQUIRES_NEW: luo aina uuden transaktion

rollbackFor

Määrittelee, minkä poikkeusten kohdalla tehdään rollback. Oletuksena rollback tehdään vain RuntimeException-tyyppisille poikkeuksille.

readOnly

Jos asetetaan true, Spring voi optimoida suoritusta, koska tiedetään, ettei tietokantaan tehdä muutoksia.